SRIM 3.2 Test Automation Architecture

Phase 1: Intake & Skill Synthesis

The lifecycle begins when a new testing requirement is introduced through a client SOW, audit scope, project closure requirement, or internal QA checklist.

Requirement Extraction

The Account Manager or Project Manager identifies the testing need from the client scope.

Example:

“Run a full security and functional audit of the client application, identify P0, P1, and P2 issues, fix approved items, generate evidence, and prepare a closure report.”

The Skill Synthesizer

The requirement is pasted into the Accomplish AI ISATVON Skill Synthesizer. The synthesizer converts the testing requirement into reusable SRIM test automation skills and maps the requirement to Dify orchestration, PostgreSQL state, FastMCP execution, MCP Jungle routing, and the required MCP tools.

ISATVON Generation

The LLM defines the exact test automation behavior:

I - Instructions:
Audit the application, classify issues, execute approved fixes, validate with tools, and generate structured reports.

S - Source:
GitHub repository, codebase folder, project files, credentials, test URL, client scope, and approval status.

A - Automation:
Dify sends an HTTP POST structured execution request to FastMCP. FastMCP then executes Python-based logic and calls MCP Jungle or direct MCP tools as required.

T - Tech Stack:
Git MCP, Filesystem MCP, TestSprite MCP, Playwright MCP, Chrome DevTools MCP, Jest, VibeShip Scanner through a FastMCP/MCP wrapper or REST adapter, Report Generator MCP, and MCP Jungle as the tool gateway.

V - Variables:
Repository URL, branch name, test environment URL, client name, project folder path, approval stage, P0/P1/P2 definitions, evidence requirements, scan rules, tool configuration, and client/project runtime variables.

O - Outcome:
Structured JSON containing audit findings, issue classification, fix status, test evidence, coverage report, security scan result, approval state, and closure status.

N - Notification:
Update Accomplish UI, PostgreSQL workflow state, audit report, approval tracker, evidence register, and closure report.

Phase 2: Database Injection & State Configuration

Once the skill definition is accepted, Accomplish stores the testing workflow configuration inside PostgreSQL.

Accept & Publish

The manager reviews the generated skill definition and clicks “Accept” in Accomplish.

PostgreSQL Update

The Accomplish backend adds the selected test automation skills into the project routing profile as JSONB.

Example routing sequence:

```
[
  "TA01_File_Scan",
  "TA02_Codebase_Audit",
  "TA03_Issue_Classification",
  "TA04_Report_Consolidation",
  "TA05_Recommendation_Review_Approval",
  "TA06_TestSprite_Fix_Execution",
  "TA07_Final_VibeShip_Security_Scan",
  "TA08_Closure_Report"
]
```

Business Rules Stored

Client-specific rules are saved in PostgreSQL.

Examples:

```
{
  "client_name": "Viva Career Academy",
  "project_type": "Web Application Testing",
  "repository_url": "GitHub URL",
  "test_environment": "Production/Staging",
  "p0_definition": "Go-live blocker",
  "p1_definition": "Important but not blocking",
  "p2_definition": "Future sprint improvement",
  "evidence_required": true,
  "video_required": true
}
```

Phase 3: Trigger & DMR Orchestration Loop

The testing workflow starts when the project files, GitHub repository, or application URL is submitted through Accomplish.

Project Intake

The internal team uploads or connects:

- GitHub repository
- Local project folder
- Website/application URL
- Test credentials
- Client scope document
- Existing issue list, if available

DMR Awakening

Accomplish triggers Dify Dynamic Master Router with:

```
{
  "project_id": "PROJECT_ID",
  "profile_id": "TEST_AUTOMATION_PROFILE_ID"
}
```

Fetch Routing Profile

Dify retrieves the test automation routing array from PostgreSQL.

Deterministic Loop

Dify executes each skill in sequence or conditional branches based on approval state and previous outputs. Dify performs orchestration, routing, skill selection, prompt decomposition,

state checks, and workflow coordination. FastMCP does not decide the overall workflow; it only executes the structured task Dify sends.

Phase 4: Audit & Reporting Execution

The first macro skill is Audit and Reporting.

This stage establishes the baseline before fixing anything.

Sub-Skill 1: File and Codebase Scan

Dify sends the scan request to FastMCP. FastMCP runs the required Python execution module and calls MCP Jungle or direct MCP tools.

Purpose:

- Detect malware
- Check unsafe files
- Identify suspicious scripts
- Validate project structure
- Confirm whether the codebase is usable for testing

Tools:

- ClamAV / Security MCP
- Filesystem MCP
- Git MCP
- MCP Jungle gateway

Sub-Skill 2: Application and Code Audit

Dify sends structured audit instructions to FastMCP. FastMCP invokes the required testing tools through MCP Jungle or direct MCP connections.

Possible tools:

- Playwright MCP
- Chrome DevTools MCP
- TestSprite MCP
- Jest adapter or Jest MCP wrapper
- VibeShip Scanner through FastMCP/MCP wrapper or REST adapter
- Dependency scanner
- Static analysis tools
- LLM-based code review only where reasoning is required

The audit identifies:

- P0 issues: mission-critical blockers
- P1 issues: important issues to fix
- P2 issues: improvements or non-blocking issues

Sub-Skill 3: Consolidated Audit Report

The findings are converted into a structured report. FastMCP returns schema-compliant JSON to Dify. Dify updates PostgreSQL and makes the report available to Accomplish.

Output includes:

```
{
  "audit_summary": "string",
  "p0_issues": [],
  "p1_issues": [],
  "p2_issues": [],
  "affected_modules": [],
  "risk_level": "High/Medium/Low",
  "recommended_next_action": "string"
}
```

Phase 5: Recommendation, Review & Approval

The second macro skill is RRA: Recommendation, Review, and Approval. No fixing starts until approval is captured.

Recommendation
Dify uses the audit output and approved prompting rules to generate recommended actions for each P0, P1, and P2 issue. FastMCP may be called only when deterministic validation or tool-based enrichment is required.

Internal Review
The manager reviews the audit report and edits recommendations if needed.

External Approval
If client approval is required, the report is shared with the client.

Internal Final Approval
After client approval, the internal team approves the final fixing scope.

Approval State Tracking
PostgreSQL tracks:

```
{
  "internal_review_status": "Pending/Approved/Rejected",
  "client_approval_status": "Pending/Approved/Rejected",
```

```
    "approved_issue_ids": [],
    "approval_date": "date",
    "approved_by": "name"
}
```

Phase 6: Fixing, Validation & Closure

The third macro skill is Fixing and Generating Updates. Approved issues are sent to TestSprite MCP or the selected fixing workflow through the FastMCP execution layer.

Fix Execution

Dify sends the approved issue scope to FastMCP. FastMCP executes the correct Python-based tool wrapper, calls MCP Jungle where needed, and returns structured results back to Dify.

Possible routes:

- TestSprite MCP for AI-assisted test generation, issue validation, and workflow-level testing support
- Jest for unit testing
- Playwright for browser testing
- Chrome DevTools MCP for frontend/debug validation
- VibeShip Scanner through MCP wrapper or REST adapter for vulnerability checks
- Git MCP for branch updates and commits
- LLM only where reasoning is required

Important Rule

Security scanning must happen after the fixes are completed, not at the beginning. Initial lightweight scanning is acceptable for baseline risk detection, but the final VibeShip/security scan must run after the fix cycle.

Coverage and Progress Reporting

After each major fix cycle, the system updates:

```
{
  "total_issues": 25,
  "fixed_issues": 18,
  "pending_issues": 7,
  "completion_percentage": 72,
  "evidence_links": [],
  "test_status": "In Progress"
}
```

Final Security Scan

Once fixes are completed, Dify triggers FastMCP to run the final security scan. FastMCP calls VibeShip Scanner through the configured MCP-compatible wrapper, REST adapter, or direct Python execution module. The result is returned as structured JSON.

Closure Report

The closure report includes:

- Original audit summary
- Approved scope
- Fixed issue list
- Pending issue list
- Test evidence
- Screenshots or videos
- Final security scan result
- Production/staging verification status
- Client acceptance section

Final Output

The Accomplish UI renders the final closure package.

Final state:

```
{
  "project_status": "Ready for Closure",
  "audit_completed": true,
  "approval_completed": true,
  "fixes_completed": true,
  "final_scan_completed": true,
  "client_acceptance_pending": true
}
```

—----------

Test Automation Prompt

Practical Example: Translating a New Test Automation Requirement

Let’s say you onboard a new client, and their SOW contains a requirement your existing test automation stack does not currently support.

SOW Excerpt

“Before production deployment, the system must automatically validate accessibility compliance against WCAG 2.1 AA standards. Any violation classified as critical must block release approval and generate an accessibility audit report.”

You do not currently have a reusable “Accessibility Compliance Validation” macro skill.

This is how the Zero-Dev SRIM 3.2 architecture handles it.

The Account Manager Uses the ISATVON Skill Synthesizer

The Account Manager pastes the SOW excerpt into the Accomplish AI Skill Synthesizer and uses the following system prompt.

SYSTEM PROMPT: ISATVON Skill Synthesizer for Test Automation

You are an Enterprise Test Automation Solutions Architect for the SRIM 3.2 platform.

Your goal is to translate unstructured Statement of Work (SOW) testing requirements into strict, reusable ISATVON skill definitions for Dify orchestration and FastMCP execution.

Execution Environment

- Accomplish AI: UI and PostgreSQL-backed state interaction
- Dify Dynamic Master Router: workflow orchestration, skill routing, prompt decomposition, and runtime coordination
- PostgreSQL: skill profiles, routing arrays, workflow state, test state, and JSONB configuration
- FastMCP: structured Python-based execution layer, tool-call wrappers, and MCP-compatible execution endpoints
- MCP Jungle Gateway: MCP tool discovery, routing, and connection layer for supported tools

---

SYSTEM RULES

- Do not place orchestration logic inside FastMCP. Dify owns orchestration.
- Do not recommend random one-off scripts. Use standardized FastMCP Python execution modules or MCP-compatible wrappers.
- Prioritize reusable MCP-compatible tools wherever possible.
- Use MCP Jungle as the preferred gateway for MCP tool discovery and routing.
- Prefer deterministic execution over LLM reasoning whenever possible.
- Only use LLM reasoning when ambiguity, semantic understanding, root cause reasoning, or decision-making is required.
- All outputs must follow strict JSON schema structures.

PREFERRED MCP TOOL PRIORITY

Prioritize mapping to the following reusable tools whenever possible:

- MCP_Jungle_Gateway
- TestSprite_MCP
- Playwright_MCP
- ChromeDevTools_MCP
- Jest_MCP_or_Jest_FastMCP_Wrapper
- Git_MCP
- Filesystem_MCP
- REST_API_MCP
- VibeShip_Scanner_FastMCP_Wrapper_or_MCP_Adapter
- AccessibilityScanner_MCP
- Postgres_MCP
- ReportGenerator_MCP
- VisualRegression_MCP
- OCR_MCP
- BrowserAutomation_MCP

EXECUTION LOGIC GUIDELINES

Use Dify orchestration when:

- Workflow routing is required
- Skill sequencing is required
- Approval gates are required
- Prompt decomposition is required
- Workflow state must be updated

Use FastMCP execution when:

- A Python-based execution module is required
- Tool-call wrappers are required
- MCP-compatible execution endpoints are required
- MCP Jungle or direct MCP tools must be called
- Structured JSON output must be returned to Dify

Use MCP tool execution when:

- The task is deterministic
- Rule-based validation exists
- API calls are involved
- Browser automation is sufficient
- Static analysis can solve the problem

Use LLM execution only when:

- Root cause analysis is required
- Semantic reasoning is needed
- UI interpretation is ambiguous
- Test recommendation generation is needed
- Natural language validation is required

OUTPUT FORMAT REQUIREMENT

Return ONLY the strict ISATVON JSON definition.

DO NOT explain the logic.
DO NOT add commentary.
DO NOT output markdown.

USER INPUT

[USER INPUT WILL BE THE TEST AUTOMATION SOW PARAGRAPH]

ISATVON TEMPLATE FOR SRIM 3.1 TEST AUTOMATION


```
{
  "Skill_Metadata": {
    "Macro_Category": "[e.g., TA01_Audit_Reporting, TA02_Security_Validation, TA03_Functional_Testing]",
    "Proposed_Skill_ID": "[e.g., TA02.4_Accessibility_Compliance_Check]",
    "Skill_Name": "[Human Readable Skill Name]",
    "Skill_Type": "[Deterministic_MCP | FastMCP_Wrapper | LLM_Assisted | Hybrid]",
    "Reusable": true
  },
  "ISATVON_Definition": {
    "I_Instructions": "[Strict execution instructions or business rules sent by Dify to FastMCP. Example: Run WCAG 2.1 AA accessibility validation against all public pages. Flag critical violations and generate a structured report.]",
    "S_Source": {
      "required_inputs": ["[e.g., project_url]", "[e.g., github_repository]", "[e.g., environment_type]"],
      "upstream_dependencies": ["[Previous completed skill IDs if required]"]
    },
    "A_Automation": {
      "orchestration_layer": "Dify_DMR",
      "execution_trigger": "HTTP POST to FastMCP",
      "tool_gateway": "MCP_Jungle",
      "execution_mode": "[Sequential | Parallel | Conditional]",
      "approval_required": "[true/false]"
    },
    "T_Tech_Stack": {
      "execution_layer": "FastMCP",
      "primary_mcp_tools": ["[e.g., AccessibilityScanner_MCP]", "[e.g., Playwright_MCP]"],
      "secondary_tools": ["[Optional fallback tools]"],
      "non_mcp_tools_wrapped": ["[e.g., VibeShip_Scanner_FastMCP_Wrapper]"],
      "llm_required": "[true/false]",
      "llm_reason": "[Only if llm_required=true]"
    },
    "V_Variables": {
      "runtime_variables": {"[key]": "[value]"},
      "client_business_rules": {"[key]": "[value]"},
      "environment_variables": {"[key]": "[value]"}
    },
```

```
    "O_Outcome": {
      "json_schema": {
        "status": "string",
        "issues_found": "integer",
        "critical_issues": "integer",
        "warning_issues": "integer",
        "affected_pages": "array",
        "report_path": "string",
        "approval_blocked": "boolean"
      },
      "success_condition": "[Define success condition]",
      "failure_condition": "[Define failure condition]"
    },
    "N_Notification": {
      "ui_updates": ["[e.g., Update Accomplish dashboard]"],
      "database_updates": ["[e.g., Append audit results to workflow/test execution state]"],
      "alerts": ["[e.g., Notify QA manager if critical issues found]"]
    }
  }
}
```

Example Output Generated by the Synthesizer

```
{
  "Skill_Metadata": {
    "Macro_Category": "TA02_Security_Validation",
    "Proposed_Skill_ID": "TA02.4_Accessibility_Compliance_Check",
    "Skill_Name": "Accessibility Compliance Validation",
    "Skill_Type": "Deterministic_MCP",
    "Reusable": true
  },
  "ISATVON_Definition": {
    "I_Instructions": "Run WCAG 2.1 AA accessibility validation against all public-facing pages. Flag any critical accessibility violations and generate structured compliance report.",
    "S_Source": {
      "required_inputs": ["project_url", "environment_type"],
      "upstream_dependencies": ["TA01.2_Website_Scan"]
    },
```

```
    "A_Automation": {
      "orchestration_layer": "Dify_DMR",
      "execution_trigger": "HTTP POST to FastMCP",
      "tool_gateway": "MCP_Jungle",
      "execution_mode": "Sequential",
      "approval_required": false
    },
    "T_Tech_Stack": {
      "execution_layer": "FastMCP",
      "primary_mcp_tools": ["AccessibilityScanner_MCP", "Playwright_MCP"],
      "secondary_tools": ["ChromeDevTools_MCP"],
      "non_mcp_tools_wrapped": [],
      "llm_required": false,
      "llm_reason": ""
    },
    "V_Variables": {
      "runtime_variables": {"wcag_standard": "WCAG_2.1_AA"},
      "client_business_rules": {"critical_threshold": "1"},
      "environment_variables": {"target_environment": "production"}
    },
    "O_Outcome": {
      "json_schema": {
        "status": "Completed",
        "issues_found": 12,
        "critical_issues": 2,
        "warning_issues": 10,
        "affected_pages": ["/home", "/checkout"],
        "report_path": "/reports/accessibility_audit.pdf",
        "approval_blocked": true
      },
      "success_condition": "Audit completed successfully",
      "failure_condition": "Accessibility scanner execution failed"
    },
    "N_Notification": {
      "ui_updates": ["Update Accomplish audit dashboard"],
      "database_updates": ["Append results to workflow_execution_state"],
      "alerts": ["Notify QA manager if critical issues exceed threshold"]
    }
  }
}
```

Core Objective of the Skill Synthesizer

The Skill Synthesizer exists to reduce dependency on developers for onboarding new test automation workflows, while still allowing structured Python execution where required through FastMCP.

Instead of random development work such as:

- Writing one-off automation scripts
- Creating hardcoded workflows
- Manually wiring every integration
- Letting execution tools decide orchestration

The platform:

- Generates reusable orchestration definitions
- Lets Dify own workflow routing and runtime coordination
- Maps execution to FastMCP modules and MCP tools
- Routes MCP tools through MCP Jungle where useful
- Stores runtime contracts in PostgreSQL
- Returns schema-compliant outputs for Accomplish UI rendering

This transforms SRIM 3.2 into a leaner orchestration system for enterprise test automation: Dify orchestrates, PostgreSQL stores state, FastMCP executes, MCP Jungle routes tools, and Accomplish displays the final result.